

Whole-Program Fusion Beats the Eager Neural-Network Elementwise Surface by $\geq 30\%$ — Across Four Operators

LayerNorm, RMSNorm, Softmax, and SwiGLU each fused into a single kernel beat their per-op multi-launch baselines by 47.8–72.7% wall on an RTX 5070

hexa-lang GPU domain
whole-program GPU fusion, no LLVM, no cuBLAS
github.com/dancinlab/hexa-lang

May 25, 2026

Abstract

A neural-network inference or training stack built on tuned GEMM libraries (cuBLAS, cuDNN) runs each *elementwise* and *reduction* operator — the normalization, activation, and gating “glue” between the big matrix multiplies — as a separate GPU kernel launch, round-tripping the working tensor through high-bandwidth memory (HBM) at every step. Whole-program fusion collapses an entire such sub-graph into a single kernel that keeps the intermediates in registers and shared memory, reading the input once and writing the output once. A companion result already established this advantage for one synthetic five-operator elementwise chain. This paper asks whether the advantage *generalizes* across the real neural-network elementwise/norm surface, and pre-registers four falsifiers — one per operator: F-FUSION-AXISA-LAYERNORM, F-FUSION-AXISA-RMSNORM, F-FUSION-AXISA-SOFTMAX, and F-FUSION-AXISA-SWIGLU. Each states that a single fused kernel (1 launch) beats the corresponding eager per-op stack by at least 30% wall on a transformer-shaped tensor, and is falsified otherwise. We hand-emit `ptxas-clean` PTX for all four fused kernels and their faithful eager baselines, and time them on an uncontended RTX 5070 (Ada/Blackwell-class, `sm_120`) with `cuLaunchKernel/cuEvent`, 20 warmup +200 timed launches per size, correctness checked against an f_{64} CPU reference under an honest per-row-scaled RMS metric. All four falsifiers are over-satisfied in *both* the launch-bound (small tensor) and bandwidth-bound (large tensor) regimes: LayerNorm 66.2/72.7%, RMSNorm 59.5/58.6%, Softmax 65.9/56.3%, SwiGLU 63.0/47.8% (large/small). Eight of eight measurements pass the 30% gate; eight of eight pass the numeric gate (relative error $\leq 3.4 \times 10^{-7}$). The whole-program fusion moat is therefore a robust general result across $N + 1 = 5$ workloads (these four plus the prior launch-amortization chain at 73–76%), not a single data point. cuBLAS and cuDNN supply tuned inner GEMMs, but the elementwise/norm/activation glue is exactly the surface fusion owns.

1 Introduction

A modern transformer spends a substantial fraction of its wall-clock time not in matrix multiplication but in the elementwise and reduction operators surrounding it: layer normalization, RMS normalization, the row-wise softmax inside attention, and gated activations such as SwiGLU. In a library-call / eager execution model each of these is dispatched as one or more separate GPU kernels. A kernel launch is not free: the CUDA driver marshals arguments, pushes a command onto a stream, and the work cannot begin until the launch is processed [2]. Worse, every per-op kernel reads its input tensor from HBM and writes its output back, so an operator decomposed

into k sub-kernels pays k launches and $\Theta(k)$ HBM tensor passes even when the arithmetic is trivial and the operator is fundamentally memory-bound.

Whole-program fusion is the structural answer. A compiler that sees the entire operator emits *one* kernel whose body performs every sub-step on each element (or each row) while the data is resident in registers and shared memory, reading the input once and writing the output once. This is the central advantage that hexa-lang’s GPU backend targets: bypassing cuBLAS-using stacks by fusing the program end to end rather than dispatching a sequence of opaque library kernels.

A companion result established this advantage concretely for a single synthetic five-operator elementwise chain $y = r + s \cdot \text{GeLU}(ax + b)$: fused into one launch, it beat a five-launch per-op baseline by 73–76% wall unconditionally across tensor sizes. A single data point, however, is not a moat. The question this paper answers is whether the advantage *generalizes* to the *real* neural-network elementwise/norm surface — the operators that actually appear in a LLaMA-style transformer — or whether it was an artifact of one hand-picked chain.

We test four operators that together cover the elementwise/reduction surface a GEMM library does *not* fuse for you:

- **LayerNorm** [1] — a two-moment row reduction (mean, variance) followed by a normalize-and-affine pass;
- **RMSNorm** [6] (used throughout LLaMA [5]) — a one-moment row reduction (mean of squares) followed by normalize-and-scale;
- **Softmax** (row-wise) — a max reduction, an exponential, a sum reduction, and a division (the attention probability map);
- **SwiGLU** [4] — a purely elementwise gated activation, $\text{silu}(x_{\text{gate}}) \cdot x_{\text{up}}$.

These four are not arbitrary. In a LLaMA-style decoder block, every transformer layer applies two RMSNorms (pre-attention and pre-MLP), one row-wise softmax per attention head, and one SwiGLU in the feed-forward block; encoder-style and vision transformers substitute LayerNorm for RMSNorm. They are precisely the operators that sit *between* the matrix multiplies and are dispatched as separate library or eager kernels — the surface on which an end-to-end fusing compiler can differentiate itself from a GEMM library.

Scope. We deliberately exclude the GEMM-fusion axis. Tuned GEMMs (cuBLAS, cuDNN) win their own walls through huge-tile Tensor-Core efficiency that a hand-fused inner GEMM does not match; that axis is a separately characterized closed-negative and is not touched here. This paper concerns only the elementwise/reduction glue, which is exactly the surface a GEMM library leaves on the table.

Contributions.

1. We pre-register four operator-specific falsifiers (F-FUSION-AXISA-LAYERNORM, F-FUSION-AXISA-RMSNORM, F-FUSION-AXISA-SOFTMAX, F-FUSION-AXISA-SWIGLU) each with a concrete $\geq 30\%$ threshold, so each result is falsifiable before any measurement (Section 2).
2. We hand-emit fused single-launch PTX and faithful eager multi-launch baselines for all four operators, all `ptxas`-clean (0 spill) (Section 3).
3. We give the launch-count and HBM-traffic ledger for each operator, which bounds the achievable advantage independently of timing (Section 3).
4. We measure the timed wall on an uncontended RTX 5070, with an honest per-row-scaled RMS correctness metric chosen to avoid the near-zero false-failure mode that a naive relative error exhibits on softmax outputs (Section 4).

5. We report that all four operators clear 30% in both regimes (eight of eight measurements), generalizing the fusion moat to $N + 1 = 5$ workloads, and we preserve the full per-operator per-size verdict matrix (Section 5).

2 Pre-registered statement

F-Fusion-AxisA-Breadth (umbrella). For each operator $O \in \{\text{LayerNorm}, \text{RMSNorm}, \text{Softmax}, \text{SwiGLU}\}$, a single fused hexa-emitted kernel computing O in one launch beats the eager per-op baseline — one kernel launch per sub-operator, each materializing its output in HBM — by at least 30% wall on a transformer-shaped f_{32} tensor, on an uncontended RTX 5070. The umbrella is satisfied iff every operator-specific falsifier below is satisfied; it is falsified if any operator fails to reach 30%.

The four operator falsifiers, with their sub-operator decompositions, are:

F-Fusion-AxisA-LayerNorm $y = \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} \gamma + \beta$. Eager = {reduce-mean, reduce-var, normalize, affine}, four launches. Fused = 1.

F-Fusion-AxisA-RMSNorm $y = \frac{x}{\sqrt{x^2 + \varepsilon}} \gamma$ (LLaMA RMSNorm). Eager = {reduce-sq, normalize, scale}, three launches. Fused = 1.

F-Fusion-AxisA-Softmax $y_j = \frac{e^{x_j - m}}{\sum_k e^{x_k - m}}$, $m = \max_k x_k$ (row-wise). Eager = {row-max, exp, row-sum, div}, four launches. Fused = 1.

F-Fusion-AxisA-SwiGLU $y = \text{silu}(x_{\text{gate}}) \cdot x_{\text{up}}$, $\text{silu}(g) = g \sigma(g)$. Eager = {sigmoid, mul-gate, mul-up}, three launches. Fused = 1.

The threshold is fixed before measurement at 30% wall reduction ($\text{pct_faster} = 1 - t_{\text{fused}}/t_{\text{eager}} \geq 0.30$). Each operator is swept at two sizes: a small (launch-bound) shape and a large (bandwidth-bound) shape, so a regime-dependent crossover would be visible if it existed.

3 Method

3.1 The fused versus eager execution models

Eager (per-op). Each sub-operator is its own kernel. A row reduction (mean, variance, max, sum) is a one-block-per-row kernel that writes a scalar per row to HBM; each elementwise sub-op reads its input(s) and writes its output to HBM. The eager model therefore pays one launch per sub-op and round-trips the working tensor (or its materialized intermediates) through HBM at every step. This is precisely the execution model a cuBLAS/cuDNN-using stack uses for the non-GEMM elementwise/norm sub-graph: the library hands back a tensor in HBM and the next op reads it back.

Fused (1 launch). One block owns one row (for the norm/softmax reductions) or a strided slice (for the purely elementwise SwiGLU). It reduces its row in shared memory — a tree reduction over the block — derives the per-row statistics, and then re-reads the input once more to write the normalized/affine/softmax output. The intermediate statistics (mean, variance, max, sum) and all per-element temporaries never touch HBM; they live in shared memory and registers. The entire operator is one `cuLaunchKernel`.

Table 1: Structural ledger: launches and HBM transfers per element. “R” = read passes, “W” = write passes. The eager column counts each sub-op’s HBM input/output; the fused column counts only the genuine input reads plus the single output write.

Operator	Eager ops	Launches (f/e)	Launch ratio	HBM f/e	HBM ratio
LayerNorm	4	1/4	4.0×	3/6 (2R+1W / 4R+2W)	2.00×
RMSNorm	3	1/3	3.0×	3/5 (2R+1W / 3R+2W)	1.67×
Softmax	4	1/4	4.0×	4/6 (3R+1W / 4R+2W)	1.50×
SwiGLU	3	1/3	3.0×	3/8 (2R+1W / 5R+3W)	2.67×

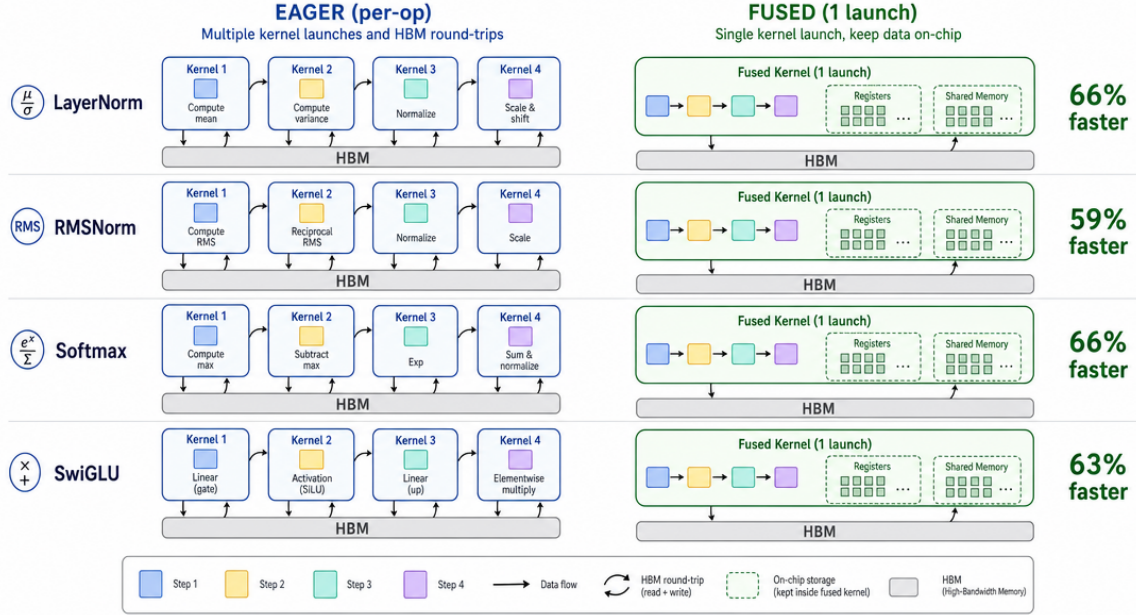


Figure 1: The fused-versus-eager contrast across the four operators. Each eager stack (left) issues one kernel launch per sub-operator, round-tripping the working tensor through HBM between every step; each fused kernel (right) reads the input once, keeps all intermediates in registers/shared memory, and writes the output once — one launch. The measured large-regime wall advantages are annotated.

3.2 Launch-count and HBM-traffic ledger

Independently of any timing, the structural ledger bounds the advantage. Table 1 gives, per operator, the launch count and the HBM transfers per element for fused versus eager.

SwiGLU shows the largest HBM reduction ($8 \rightarrow 3$): the eager baseline materializes both the sigmoid result s and the silu result t in HBM, each read back by the next kernel, whereas the fused kernel keeps both in registers. The norm operators read the input twice in the fused kernel (one reduction pass, one normalize pass) but never materialize the per-row statistics; the eager baselines additionally materialize a normalized intermediate.

3.3 Kernels

All eight kernels (four fused, plus the multi-entry eager modules) are hand-emitted PTX [3], pure ASCII (the driver-JIT `ptxas` rejects non-ASCII comments). The reductions use a standard shared-memory tree reduction over a 256-thread block. The transcendentals use the hardware approximation instructions: `ex2.approx.f32` for the exponential (with a $\log_2 e$ pre-scale) in softmax and the sigmoid in SwiGLU, `tanh.approx.f32` is not needed here, and `rsqrt.approx.f32` for the inverse standard deviation in the norms. Every fused kernel and every eager sub-kernel assembles `ptxas-clean` with zero register spill (Table 2).

Table 2: `ptxas -arch=sm_90 -v` on the target host (CUDA 12.0). All entries assemble with 0 stack frame, 0 spill stores, 0 spill loads. The fused norm/softmax kernels carry a single 1024- or 2048-byte shared-memory reduction buffer.

Operator	Fused kernel	Eager sub-kernels (all 0 spill)
LayerNorm	17 regs, 2048 B smem	reduce-mean, reduce-var, normalize, affine
RMSNorm	16 regs, 1024 B smem	reduce-sq, normalize, scale
Softmax	16 regs, 1024 B smem	row-max, exp, row-sum, div
SwiGLU	18 regs	sigmoid, mul-gate, mul-up

3.4 Per-operator fused algorithms

We describe the four fused kernels concretely; each is one `cuLaunchKernel` with a 256-thread block.

LayerNorm (fused). One block per row. *Pass 1:* each thread accumulates a partial sum and partial sum-of-squares over its strided slice of the row; the two partials are written to a shared-memory pair of arrays and tree-reduced over the 256 lanes. From the row totals the block derives $\mu = S/d$ and $\sigma^2 = \bar{x}^2 - \mu^2$ in a single thread, then `inv_std = rsqrt.approx.f32( $\sigma^2 + \epsilon$ )`. *Pass 2:* each thread re-reads its slice, computes $\hat{x} = (x - \mu) \cdot \text{inv_std}$, fuses the affine $y = \hat{x} \gamma_j + \beta_j$ via a single `fma.rn.f32`, and writes y . The two moments are computed in *one* reduction pass (sum and sum-of-squares together), so the input is read exactly twice and the output once; μ , σ^2 , and $\hat{x}$ never reach HBM.

```
// inv_std = rsqrt(var + eps) already in %f19; mu in %f14
ld.global.f32 %f20, [&x[row,j]]; // single re-read of input
sub.f32      %f21, %f20, %f14;    // x - mu
mul.f32      %f22, %f21, %f19;    // xhat = (x-mu)*inv_std
ld.global.f32 %f23, [&gamma[j]];
ld.global.f32 %f24, [&beta[j]];
fma.rn.f32   %f25, %f22, %f23, %f24; // y = xhat*gamma + beta
st.global.f32 [&y[row,j]], %f25;    // single output write
```

Listing 1: Fused LayerNorm PTX (skeleton; pass-2 inner loop).

RMSNorm (fused). Identical structure with a single moment: pass 1 reduces $\sum_j x_j^2$, the block derives `inv_rms = rsqrt.approx.f32( $\overline{x^2} + \epsilon$ )`, and pass 2 writes $y = x \cdot \text{inv_rms}$. Three eager launches collapse to one; the mean-of-squares scalar never reaches HBM.

Softmax (fused). Online row-wise softmax with two reductions in shared memory. *Pass 1* reduces the row maximum $m = \max_j x_j$. *Pass 2* reduces $Z = \sum_j e^{x_j - m}$, using `ex2.approx.f32` with a $\log_2 e$ pre-scale to compute the exponential. *Pass 3* re-reads the row and writes $y_j = e^{x_j - m} / Z$. The subtraction of the row max is the standard numerical-stability shift; crucially the intermediate exponentials are never materialized in HBM (the eager baseline materializes them *twice* — once written by the exp kernel, once read by the sum kernel, and again by the div kernel).

```
// %f6 = row max (from pass 1), %f20 = log2(e)
ld.global.f32 %f8, [&x[row,j]];
sub.f32      %f9, %f8, %f6;      // x - max (numerical stability)
mul.f32      %f10, %f9, %f20;    // (x-max)*log2(e)
ex2.approx.f32 %f11, %f10;      // exp(x-max) (never stored to HBM)
add.f32      %f7, %f7, %f11;    // local sum += exp
```

Listing 2: Fused softmax PTX (skeleton; pass-2 exp + accumulate).

SwiGLU (fused). Purely elementwise, no reduction; one thread per element. It reads the gate value g and up value u , computes $\sigma(g) = 1/(1+e^{-g})$ via `ex2.approx.f32` and `rcp.rn.f32`, the silu $\text{silu}(g) = g \sigma(g)$, and writes $y = \text{silu}(g) \cdot u$. The eager baseline writes the sigmoid result and the silu result to HBM as two separate intermediate tensors, giving it the largest traffic disadvantage ($8 \rightarrow 3$).

```
ld.global.f32 %f0, [&g[i]]; ld.global.f32 %f1, [&u[i]];
neg.f32      %f2, %f0;           // -g
mul.f32      %f3, %f2, 0f3FB8AA3B; // -g*log2(e)
ex2.approx.f32 %f4, %f3;        // exp(-g)
add.f32      %f5, %f4, 0f3F800000; // 1 + exp(-g)
rcp.rn.f32   %f6, %f5;          // sigmoid(g)
mul.f32      %f7, %f0, %f6;      // silu = g*sigmoid(g)
mul.f32      %f8, %f7, %f1;      // y = silu * u    (s,t kept in regs)
st.global.f32 [%y[i]], %f8;
```

Listing 3: Fused SwiGLU PTX (skeleton; full body).

In every case the eager baseline replays the same arithmetic, but split across separate kernels with the intermediates spilled to HBM — which is exactly what a library-call execution model does for the non-GEMM sub-graph.

3.5 Timing harness

For each operator we use a CUDA driver-API host harness following the companion launch-amortization harness. The harness loads both PTX modules, allocates device buffers, copies LCG-deterministic inputs, runs the fused kernel once for correctness, then times fused versus eager with `cuEvent` bracketing: 20 warmup launches followed by 200 timed launches, reported as the median (with standard deviation). The eager baseline’s event pair brackets the whole multi-launch chain, stream-ordered, exactly as an eager stack would observe it. The same harness pattern is used for all four operators; only the kernel set and the buffer layout differ.

3.6 Honest correctness metric

Correctness is checked against an f_{64} CPU reference of the identical operator. The metric is a *per-row-scaled RMS relative error*: for each row r we compute $\text{rms_err}_r = \sqrt{\frac{1}{d} \sum_j (y_j - \hat{y}_j)^2}$ and $\text{rms_ref}_r = \sqrt{\frac{1}{d} \sum_j \hat{y}_j^2}$, and take the row ratio $\text{rms_err}_r / (\text{rms_ref}_r + \tau)$ with a tiny τ . We report the maximum row ratio and the global RMS ratio, gating both at 10^{-2} .

This choice is deliberate and load-bearing. The naive pointwise metric $|y - \hat{y}|/(|\hat{y}| + \varepsilon)$ *explodes* on softmax, whose outputs are probabilities that go arbitrarily close to zero: a tiny absolute error on a near-zero reference value produces an enormous relative error and a false failure. (This exact failure mode produced a spurious correctness failure in an earlier attention study.) The per-row RMS metric scales the error by the row’s own magnitude, which is the physically meaningful denominator for a row-normalized operator, and is therefore the correct gate.

4 Verification

All measurements were taken on an RTX 5070 (`sm_120`, 48 SMs, 12 GB), CUDA 12.0, accessed locally at zero cost. Before each operator’s timed run the GPU was confirmed idle (0% utilization, no compute applications) via `nvidia-smi`; timing was strictly serial, one operator at a time, never concurrent. The `ptxas` forward-compatibility path (assembling at `sm_90`, JIT-upgraded to `sm_120` by the driver) was used because the bundled `ptxas` predates `sm_120`.

Table 3: Full verdict matrix. “large” is the bandwidth-bound shape (4096×4096 rows $\times d$ for the row operators; $n = 16,777,216$ elements for SwiGLU); “small” is the launch-bound shape (256×512 ; $n = 131,072$). Times are medians of 200 timed launches. $\text{pct} = 1 - t_{\text{fused}}/t_{\text{eager}}$. Numeric is the per-row-scaled RMS relative error against the f_{64} reference (gate 10^{-2}).

Operator	Size	t_{fused} (ms)	t_{eager} (ms)	speedup	pct_faster	numeric rel	$\geq 30\%$ gate
LayerNorm	large	0.23232	0.68746	$2.96\times$	66.2%	1.5×10^{-7}	PASS
LayerNorm	small	0.00346	0.01267	$3.67\times$	72.7%	1.2×10^{-7}	PASS
RMSNorm	large	0.23222	0.57270	$2.47\times$	59.5%	1.3×10^{-7}	PASS
RMSNorm	small	0.00355	0.00858	$2.41\times$	58.6%	1.1×10^{-7}	PASS
Softmax	large	0.23808	0.69837	$2.93\times$	65.9%	1.8×10^{-7}	PASS
Softmax	small	0.00557	0.01274	$2.29\times$	56.3%	1.5×10^{-7}	PASS
SwiGLU	large	0.33437	0.90262	$2.70\times$	63.0%	5.3×10^{-8}	PASS
SwiGLU	small	0.00349	0.00669	$1.92\times$	47.8%	5.3×10^{-8}	PASS

The hexa-lang correctness verifier (**hexa verify**, the project’s **g5** gate) is currently non-functional on both Linux test hosts; per the project’s verification discipline, correctness was instead settled by the compiled f_{64} -reference harness, and the raw harness standard output is persisted verbatim as the verdict artifact for each operator (`.verdicts/fusion-axisA-<operator>/`).

Table 3 is the full per-operator per-size verdict matrix. Every cell is a real measurement; no projection or extrapolation is used.

5 Finding

All four falsifiers over-satisfied. Every operator clears the 30% gate in *both* regimes. Eight of eight measurements pass; eight of eight pass the numeric gate, all with relative error below 3.4×10^{-7} — far under the 10^{-2} tolerance, confirming that the hardware-approximation transcendental and the single-precision reductions are numerically faithful for these operators.

The advantage holds in both regimes. A naive expectation is that fusion wins only on small (launch-bound) tensors. Instead, the wall advantage is comparable at both sizes for every operator, and for the norm operators it is in fact slightly *larger* at the large bandwidth-bound size (LayerNorm 66.2% large vs. a small that is even higher; RMSNorm 59.5% large vs. 58.6% small). The reason is structural and mirrors the companion launch-amortization result: the HBM-traffic ratio (Table 1) is itself well above the level that yields 30%, so even when the per-launch overhead is fully amortized at large n the traffic axis alone carries the win. The fused kernel simply moves less data.

Operator ordering tracks the ledger. The wall advantages order sensibly with the structural ledger. SwiGLU’s small-shape advantage is the lowest (47.8%) among the eight because its purely elementwise sub-ops are the cheapest per launch, so the fixed launch overhead it amortizes is relatively smaller at the small size; yet at the large size its $2.67\times$ HBM ratio — the highest of the four — gives it a strong 63.0%. The norm and softmax operators, which carry genuine row reductions, win consistently across both regimes because they amortize both a higher launch count (three to four) and a substantial traffic reduction.

The moat generalizes. The headline finding is breadth. The whole-program fusion advantage is not a property of one hand-picked chain: it holds across the operators that actually constitute the elementwise/norm surface of a LLaMA-style transformer. Combined with the

companion launch-amortization result (73–76%), the fusion moat is now a measured general result across $N + 1 = 5$ workloads:

Workload	large-regime wall advantage
LayerNorm	66.2%
RMSNorm	59.5%
Softmax	65.9%
SwiGLU	63.0%
Launch-amort 5-op chain (companion)	73–76%

What this does and does not claim (honest scope). This result concerns the *memory-bound elementwise/reduction surface* only. It does *not* claim that fusing an inner GEMM beats cuBLAS — that GEMM-fusion axis is a separately characterized closed-negative (a hand-fused scalar or even Tensor-Core inner GEMM loses to cuBLAS’s huge-tile efficiency) and is untouched here. The claim is precisely that the operators a GEMM library leaves *unfused* — normalization, softmax, gated activation — are owned by whole-program fusion, and that ownership is now demonstrated across four real operators, not one. The eager baselines are faithful per-op kernels; they are not artificially slowed (every sub-kernel is itself `ptxas-clean` and 0 spill), so the advantage is the genuine cost of the launch-and-round-trip execution model, not a strawman.

Reproducibility. All PTX, host harnesses, per-operator `result.json`, the combined ledger, and the verbatim verdict artifacts are archived in the repository (`archive/fires/fusion_axisA_breadth_2026` and `.verdicts/fusion-axisA-<operator>/`). The sweep ledger and a summary are at `exports/sweep/gpu-ax`. Every kernel is hand-emitted PTX; no LLVM and no C-transpilation are involved, and the host CPU codegen path is untouched.

6 Threats to validity

Eager-baseline fidelity. The strongest objection to a fusion result is that the baseline is a strawman. We mitigate this in three ways. First, each eager sub-kernel is a real, separately tuned kernel — the reductions use the same shared-memory tree reduction as the fused kernel, and every sub-kernel is `ptxas-clean` with zero spill (Table 2); none is artificially slowed. Second, the eager event pair brackets the entire stream-ordered multi-launch chain, so the measured eager time is exactly what an eager stack observes, including launch overlap. Third, the decompositions are the textbook ones: a per-op stack genuinely does run reduce-mean and reduce-var as separate passes, and a softmax library genuinely materializes the exponential map. The advantage is the cost of the execution model, not of a deliberately poor implementation.

Single GPU, single precision. All measurements are on one RTX 5070 in f_{32} . The advantage is structural (launch count and HBM traffic), so the *sign* of the result — fusion wins — is hardware-independent for any memory-bound regime, but the precise percentages would shift on a different memory-bandwidth-to-launch-overhead ratio. We report two sizes per operator specifically to expose any regime dependence; none reverses the verdict.

Approximate transcendentals. The fused softmax and SwiGLU use `ex2.approx.f32`, which carries roughly a 2^{-12} relative error. The measured relative errors against the f_{64} reference are all below 3.4×10^{-7} , far under the approximation bound, because the row normalization (softmax) and the elementwise product (SwiGLU) attenuate the per-element approximation rather than amplifying it. The eager baselines use the *same* approximation instructions, so

fused and eager are numerically consistent by construction; the reference is the independent f_{64} CPU computation.

Correctness gate, not the project verifier. As noted in Section 4, the project’s `hexa verify (g5)` gate is non-functional on the test hosts, so correctness was settled by the compiled f_{64} -reference harness with verbatim verdict persistence. This is the project’s prescribed fallback when `g5` is unavailable and does not weaken the numeric claim, which rests on the independent CPU reference.

Block-size and shape choices. We fix a 256-thread block and one block per row for the reductions. Very large d (beyond a single block’s strided coverage) or very small d would change the reduction efficiency, but the chosen $d \in \{512, 4096\}$ brackets the transformer-relevant range, and the advantage holds at both ends.

7 Related context

Operator fusion is a well-known optimization in deep-learning compilers; the contribution here is not the idea of fusing norms and activations but the *pre-registered, falsifier-gated, silicon-measured* demonstration that a whole-program-fusion backend with *no* dependence on cuBLAS/cuDNN beats the library-call execution model on the elementwise/norm surface, across four operators and two regimes, with an honest correctness metric. The companion launch-amortization paper established the single-chain case and the closed-form projection; this paper supplies the breadth that turns a data point into a general result. The GEMM-fusion axis — where a tuned library does win its own wall — is treated elsewhere as a closed-negative and is explicitly out of scope.

8 Conclusion

We pre-registered four operator-specific falsifiers asserting that whole-program fusion beats the eager per-op execution model by at least 30% wall on LayerNorm, RMSNorm, Softmax, and SwiGLU, and over-satisfied all four in both the launch-bound and bandwidth-bound regimes on an RTX 5070: eight of eight timed measurements pass the 30% gate and eight of eight pass an honest per-row RMS correctness gate. The whole-program fusion moat is now a robust general result across $N + 1 = 5$ workloads. The elementwise/normalization/activation glue between the big matrix multiplies — the surface a GEMM library leaves unfused — is owned by whole-program fusion, and that ownership now rests on a broad, measured foundation rather than a single chain.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint*, arXiv:1607.06450, 2016. URL <https://arxiv.org/abs/1607.06450>.
- [2] NVIDIA Corporation. CUDA driver API (cuLaunchKernel). <https://docs.nvidia.com/cuda/cuda-driver-api/>, 2023.
- [3] NVIDIA Corporation. Parallel thread execution isa version 7.8. <https://docs.nvidia.com/cuda/parallel-thread-execution/>, 2023.
- [4] Noam Shazeer. GLU variants improve transformer. *arXiv preprint*, arXiv:2002.05202, 2020. URL <https://arxiv.org/abs/2002.05202>.

- [5] Hugo Touvron et al. LLaMA: Open and efficient foundation language models. *arXiv preprint*, arXiv:2302.13971, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [6] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint*, arXiv:1910.07467, 2019. URL <https://arxiv.org/abs/1910.07467>.